

It Is Not the Cryptography: Where the Cost of Token Validation Actually Goes in Cloud-Native Microservices

Katha Sikdar¹

¹Independent Researcher.

Contributing authors: ferdouskatha35@gmail.com;

Abstract

The runtime cost of API security in microservices is routinely attributed to cryptographic work. We show that this attribution is wrong by two orders of magnitude. Measuring JSON Web Token validation inside a running Node.js service under layered Zero Trust enforcement — edge TLS, Istio mutual TLS and application-level validation — we find that the signature primitive accounts for under one percent of the per-request cost. Approximately 76% of the cost at the arrival rates typical of published evaluations is key material re-converted from a string on every call, an avoidable defect that a one-line change removes. In isolation on the host, a single HS256 validation costs 26.01 μ s; inside the deployed container the same call costs 435.72 μ s with a string secret and 8.26 μ s with a pre-parsed key object, a factor of 52.8. We further show that per-call cost depends strongly on arrival rate — rising from 39.38 μ s at 400 requests per second to 221.42 μ s at 5 — and that this is neither just-in-time compilation warmth nor garbage collection but the inter-arrival gap itself. Because in-situ duration distributions can only be inflated by host contention, these findings are conservative. We also report that relocating validation into the service mesh, the remedy our own earlier work proposed, duplicates rather than moves the work unless the application trusts a sidecar-populated header, and that doing so opened an authentication bypass in our testbed which we demonstrate and close. The practical consequences are that keys must be pre-parsed, that algorithm selection must not be driven by primitive benchmarks, and that offloading is an architectural choice with a trust-boundary cost rather than a remedy for this overhead.

Keywords: microservices, service mesh, JSON Web Token, performance measurement, Zero Trust, benchmarking methodology

1 Introduction

A recurring claim in the literature on cloud-native security is that authentication imposes a measurable per-request cost, and that this cost is cryptographic. The claim is intuitive: validating a JSON Web Token involves verifying a signature, signatures involve cryptography, and cryptography is expensive relative to parsing. It also has an appealing consequence, namely that the cost can be moved — to a sidecar proxy, to an API gateway, to dedicated hardware — because cryptographic work is the kind of work that specialised components do well.

We report that the claim is wrong, and wrong by a wide margin, for the most widely deployed configuration we could measure. In a Node.js service under edge TLS, Istio mutual TLS and application-level HS256 validation, the signature primitive accounts for 5.7% of the cost of the whole `jwt.verify()` call measured in isolation, and under one percent of the per-request cost measured in deployment. The dominant term is not cryptography and not parsing. It is the conversion of key material from a string into a key object, performed once per call because the application passes a string, which is what the library’s interface invites and what published testbeds — ours included — have done.

This distinction matters for three reasons. First, it changes the remedy. A cost that is cryptographic is a candidate for offloading; a cost that is repeated type conversion is a candidate for being deleted. Second, it changes how algorithms should be compared. Measured on primitives alone, RS256 is far more expensive than HS256; measured end to end in deployment, the difference is much smaller, because a fixed per-call overhead

dominates both. Third, it is a methodological warning. The gap we report is invisible to a microbenchmark run on a developer machine, and our own isolated measurements understated the deployed cost by more than an order of magnitude.

Contributions

We make the following contributions, all supported by measurements in the replication package [Sikdar \(2026b\)](#).

1. An in-situ measurement of token validation inside a running service, using a histogram that times the verification call alone, rather than inferring its cost by differencing scenario averages (Section 3).
2. A decomposition showing that the signature primitive is a single-digit percentage of the isolated call and under one percent of the deployed cost, and that key conversion is the dominant term (Section 4).
3. Evidence that per-call cost depends on arrival rate, with an ascending and descending sweep that rules out just-in-time warmth, and a controlled experiment isolating the inter-arrival gap as the mechanism (Section 4).
4. A like-for-like algorithm comparison with both algorithms on pre-parsed key material, showing that primitive benchmarks mis-rank deployment cost (Section 4).
5. A security result: mesh-based offloading duplicates validation unless the application trusts a sidecar-populated header, and trusting that header opened an authentication bypass, which we demonstrate, map to API2:2023 [OWASP Foundation \(2023\)](#), and close (Section 5).

6. A correction to our own earlier study [Sikdar \(2026a\)](#), whose per-request attribution this paper supersedes (Section 2).

2 Background and related work

2.1 Layered enforcement and sidecar overhead

Cloud-native deployments commonly stack several independent enforcement mechanisms: TLS at the edge, mutual TLS between workloads, and application-level token validation. Each is administered separately, and their costs are usually reported separately.

Sidecar overhead is the best-characterised of the three. [Zhu et al. \(2023\)](#) decompose service mesh sidecar cost and show that the dominant contributors depend on configuration, with inter-process communication and socket writes dominating for TCP proxying and protocol parsing dominating for HTTP proxying. Their central methodological point — that the overhead is tied to configuration and workload rather than being a fixed tax — is one we reach independently for validation cost. [Bremner Barr et al. \(2024\)](#) compare mutual TLS across Istio, Istio Ambient, Linkerd and Cilium, attributing differences to sidecar versus sidecarless architecture. [Singh et al. \(2025\)](#) evaluate Istio’s effectiveness in Kubernetes, and [Hanada and Ishibashi \(2025\)](#) address latency and failure trade-offs in microservice call chains. None of these isolate application-level token validation, which is the layer we measure.

2.2 Token validation cost

Compared with mesh overhead, the cost of token validation itself is thinly evidenced. It is frequently reported as a difference between scenario averages — a configuration with validation enabled minus one without — which attributes to validation every difference between the two runs, including scheduling, queueing and measurement noise. Our own earlier study [Sikdar \(2026a\)](#) did exactly this, and Section 4 shows what that approach captured.

2.3 Zero Trust in cloud-native systems

Zero Trust replaces perimeter trust with continuous verification of every request. Recent systematic reviews by [Gambo and Almulhem \(2025\)](#) and [Mushtaq et al. \(2025\)](#) survey its architecture and adoption barriers. The relevant consequence for performance work is that verification becomes per-request and unavoidable, so its cost is paid on every call rather than once per session — which is precisely why the per-call figure, and its correct attribution, matters.

2.4 Event-driven runtimes

Node.js executes application logic on a single thread, so any synchronous per-request work occupies the event loop and delays every other pending request. [Kabamba et al. \(2026\)](#) develop runtime tracing for single-threaded environments, and [Tymoshenko et al. \(2026\)](#) argue that CPU-based autoscaling signals do not capture event loop saturation for Node.js workloads. [Peng et al. \(2025\)](#) address performance prediction and adaptive resource adjustment for cloud-native

microservices more generally. Our instrumentation follows this line in measuring event loop delay directly, but our claim concerns where per-call time goes rather than capacity.

2.5 Microbenchmarks versus production behaviour

This is the literature our central result speaks to most directly, and it was absent from our earlier work. Schiavio et al. [Schiavio et al \(2026\)](#) show that microbenchmarks executed in isolation can cause a managed runtime to collect unrealistic execution profiles, producing aggressive optimisations that would not occur in a real application, and therefore misleading results even when established guidelines are followed. Traini et al. [Traini et al \(2022\)](#) show that steady state is frequently not reached at all, and that developers routinely misjudge the end of the warmup phase. Our results are an instance of exactly this hazard in a different runtime: the same call, on the same machine, costs $9.63 \mu s$ in a tight loop and $233.25 \mu s$ when invoked at realistic intervals.

2.6 Cryptographic library performance

Work on cryptographic cost in protocols concentrates on handshake and key exchange. Montenegro et al. [Montenegro et al \(2026\)](#) compare post-quantum handshake performance across QUIC and TLS, and Souvatzidaki and Limniotis [Souvatzidaki and Limniotis \(2025\)](#) analyse post-quantum key exchange in TLS 1.3. These measure primitives and protocol exchanges carefully. Our finding is complementary and cautionary: in an application-level validation path, the primitive is not what dominates, so

primitive-level measurement is a poor predictor of deployed cost.

2.7 Relationship to our earlier study

This paper corrects [Sikdar \(2026a\)](#). That study reported a per-request validation cost derived by differencing amortised pod CPU between a full-stack scenario and a mutual-TLS-only scenario, and attributed the difference to signature verification. The present work measures the validation call directly rather than by differencing, and finds that the attribution was incorrect: the signature primitive is a negligible fraction of the cost, and the dominant term is per-call key conversion. We state the correction explicitly in Section 4 and do not reuse any figure from that study.

3 Methodology

3.1 Testbed

The testbed is a single-node Kubernetes cluster (v1.32.2) running Istio with automatic sidecar injection, an injected NGINX ingress controller terminating edge TLS, and two Node.js services. Requests traverse edge TLS, the ingress proxy, a mutual TLS leg between sidecars, and the application. The application under test exposes a single JSON endpoint and validates a bearer token on every request. Node.js is v26.6.0. Full manifests, image digests and per-run provenance are in the replication package [Sikdar \(2026b\)](#).

The service selects its behaviour from environment variables rather than from separate builds, so that every configuration we compare runs from one image and one digest: `PSAO_AUTH_MODE` enables or disables validation, `PSAO_JWT_ALG` selects

HS256 or RS256, and `PSAO_JWT_KEYFORM` selects whether key material is passed as a string or pre-parsed once at startup. Comparing configurations built from different images would place build differences inside the comparison.

3.2 Instrumentation

Two instruments matter here. The first is a histogram that times the verification call alone, wrapping it with a monotonic high-resolution clock and recording the duration under a label naming the algorithm in force. This is the decisive instrument: it measures the call rather than inferring its cost from a difference between scenarios. The second is an event loop delay monitor at one-millisecond resolution, together with a request duration histogram covering the whole handler. Metrics are exposed on a port separate from the application port, so that scraping does not compete with the load under test.

3.3 Isolated microbenchmark

For comparison against in-situ measurement we decompose `jwt.verify()` into four stages, measured on the host outside any container: the full call; structural decoding only, which splits the token and parses the header and payload without checking any signature; the signature primitive with the key already parsed outside the measurement loop; and the primitive over raw buffers. The residual — the full call minus decoding minus primitive — is library dispatch, key handling, allocation and claim validation. Each stage runs 180000 iterations after a warmup, at three payload sizes, for both HS256 and RS256.

Repeatability is reported rather than assumed. Across three passes, every

`full_verify` figure reproduced within 2.7%. Sub-microsecond stages are less stable and we do not quote them to three significant figures.

3.4 In-situ rate sweep

The central experiment drives the deployed service at a series of fixed arrival rates — 5, 10, 25, 50, 100, 200 and 400 requests per second — for three minutes per point, and reads the verification histogram before and after each point. Because the histograms are cumulative, differencing the two readings isolates the distribution for that rate alone without restarting the process.

Two design choices deserve statement. First, the process is warmed before the first point, so that the lowest rate is not also the first traffic the process has ever served. Second, the sweep runs ascending and then descending on the same process. A single ascending pass confounds arrival rate with accumulated just-in-time warmth, because the highest rate would benefit from every earlier point's compilation; a descending pass that reproduces the ascending curve rules that out.

3.5 Why in-situ duration is robust to contention

The load generator shares a host with the cluster, and host load varied across our runs. This would be fatal to a throughput or saturation measurement, because contention reduces achievable throughput. It is not fatal here. We measure a duration distribution of a CPU-bound call, and contention can only *inflate* such a measurement. Every in-situ figure we report is therefore an upper bound, and the central finding — that deployed cost greatly exceeds isolated cost — is conservative

under contention rather than produced by it. Host load is recorded per point in the replication package so this can be checked.

3.6 Key form and algorithm comparisons

To separate the effect of key handling from the effect of the algorithm, we measure all four combinations of algorithm and key form inside the deployed container in a single process, with 40000 iterations each. We then repeat the full rate sweep with HS256 on a pre-parsed key, so that the HS256 and RS256 sweeps differ only in algorithm.

4 Results

4.1 Isolated cost and its decomposition

Measured on the host in a tight loop, a single HS256 validation of a 0.5 kB token costs 26.01 μ s. Of this, structural decoding is 0.895 μ s and the signature primitive is 1.484 μ s, leaving a residual of 23.63 μ s. The primitive is 5.7% of the call and the residual is 90.9%. For RS256 the full call is 29.00 μ s with a primitive of 11.159 μ s.

Even in isolation, then, the cryptographic primitive is a single-digit percentage of HS256 validation. Figure 1 shows the decomposition.

4.2 In-situ cost greatly exceeds isolated cost

Measured inside the running service at 200 requests per second, the same HS256 validation costs 421.21 μ s — a factor of approximately 16.2 above the isolated figure. No observation fell in the lowest histogram buckets; the isolated cost is not attained at any percentile.

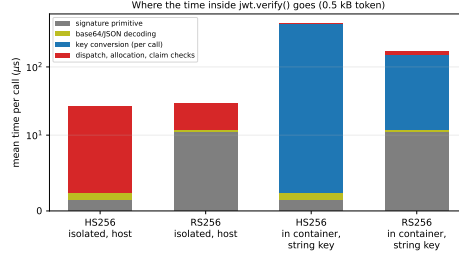


Fig. 1 Where time inside `jwt.verify()` goes, for a 0.5 kB token. The signature primitive is a small fraction in every configuration; in the deployed container with a string key, per-call key conversion dominates.

Table 1 Mean cost of one validation inside the deployed container, by algorithm and key form (40000 iterations each).

Algorithm	Key form	Mean (μ s)
HS256	string secret	435.72
HS256	pre-parsed object	8.26
RS256	PEM string	171.80
RS256	pre-parsed object	31.99

This is real processor time, not scheduling delay. With validation disabled and all other work identical, application CPU at 200 requests per second was 69.45 millicores; with validation enabled it was 146.38 millicores. The difference of 76.92 millicores corresponds to 384.6 μ s of CPU per request, consistent with the wall-clock histogram.

4.3 Key conversion is the dominant term

Measured inside the deployed container, with identical cryptography and an identical token, changing only how the key reaches the library:

Pre-parsing reduces HS256 validation by a factor of 52.8, removing 98.1% of the cost, and RS256 by a factor of 5.4. The effect is larger for HS256 than for RS256: converting a string secret into a

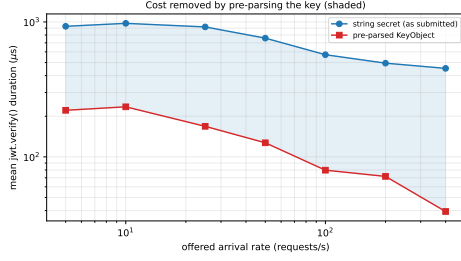


Fig. 2 Validation cost with a string secret and with a pre-parsed key object, across arrival rates. The shaded region is cost removed by a one-line change.

keyed-hash key is more expensive here than parsing a PEM.

Across the rate sweep the same effect holds end to end. At 5 requests per second, mean cost falls from 928.24 μ s with a string secret to 221.42 μ s with a pre-parsed key, a saving of 706.82 μ s, or 76% of the total. At 400 requests per second the saving is 91%. Figure 2 shows the two curves.

4.4 Cost depends on arrival rate

Per-call cost falls monotonically with arrival rate. With a pre-parsed key it falls from 221.42 μ s at 5 requests per second to 39.38 μ s at 400. The descending pass reproduces the ascending pass within 10.7% at every point, which rules out accumulated just-in-time warmth: by the descending pass the process had already served every higher rate, and the low-rate points remained slow.

Garbage collection is also ruled out. Collection time rises with arrival rate while per-call cost falls, and collection accounts for a small fraction of verification time throughout.

The mechanism is the inter-arrival gap itself. In a single warmed process with no HTTP, no server and no contention, inserting an idle interval between

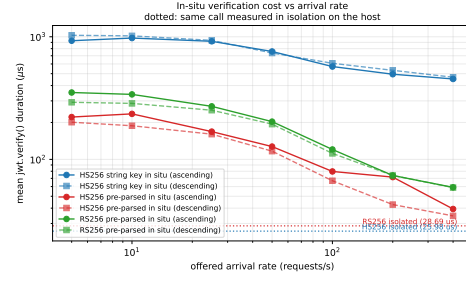


Fig. 3 In-situ validation cost against arrival rate, with the isolated microbenchmark values as horizontal references. Ascending and descending passes are drawn separately; their agreement rules out just-in-time warmth.

successive calls reproduces the effect: with a pre-parsed key, cost rises from 9.63 μ s in a tight loop to 233.25 μ s at a 200 ms gap, a factor of 24.2. Caches and branch predictors do not survive the gap. Figure 3 plots the in-situ curves against the isolated microbenchmark values.

4.5 Algorithm choice looks different in deployment

On primitives, RS256 is far more expensive than HS256: 11.159 μ s against 1.484 μ s. In the deployed container with both algorithms on pre-parsed keys, the whole call differs by a factor of 3.9. Across the rate sweep the ratio is smaller still, around 1.59 at 5 requests per second and 1.50 at 400. Figure 4 contrasts the two views.

4.6 How much of observed latency is the application

At 200 requests per second, mean client-observed latency was 1944 μ s, while the server-side handler histogram recorded 548.98 μ s. The application accounts for approximately 28% of what the client sees; the remainder is TLS, the ingress, two proxy hops, and the load generator

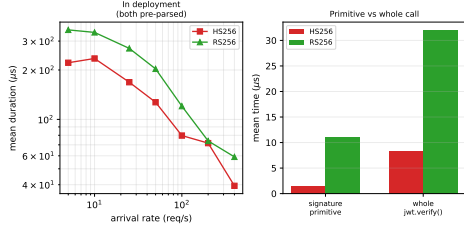


Fig. 4 RS256 against HS256, in deployment and on primitives. A primitive-level comparison overstates the deployed difference, because fixed per-call overhead dominates both.

competing for the same host. Any attribution of per-request cost derived from client-side latency inherits that.

4.7 What this means for the earlier attribution

Our earlier study [Sikdar \(2026a\)](#) reported a per-request validation cost obtained by differencing scenario CPU, and attributed it to signature verification. The measurements here show that under one percent of validation cost is the signature primitive, and that the large majority at the arrival rates that study used is per-call key conversion. Its service passed a string secret to the validation library on every request, which is the configuration measured in the first row of Table 1. The magnitude it reported is broadly consistent with what we measure for that configuration; the attribution is not.

5 Offloading, and the bypass

Because the earlier work proposed relocating validation into the sidecar, we implemented that policy and measured it. The security findings are more interesting than the performance ones.

5.1 A mesh policy duplicates rather than relocates

Applying an Istio `RequestAuthentication` causes the sidecar to validate the token before proxying the request. It does not cause the application to stop validating. Unless the handler is changed, validation now happens twice and no work leaves the event loop. Relocation therefore requires the application to trust something the sidecar tells it — in our implementation, a header into which the sidecar writes the verified claims — and to skip its own signature check when that header is present.

5.2 The bypass

That trust is the vulnerability. The sidecar overwrites the header on every request, so a client cannot forge it *while the policy is applied*. But the policy is applied only transiently, when the controller decides to offload. In every other state nothing sanitises the header. With no policy applied and no `Authorization` header at all, a request carrying a hand-written claims header was served with the protected payload: a complete authentication bypass, corresponding to API2:2023 Broken Authentication [OWASP Foundation \(2023\)](#).

5.3 Closing it

We close the bypass with an always-applied Envoy filter that strips any client-supplied copy of the header on entry to the pod, before any validation filter runs. It is matched on the connection manager rather than on the validation filter, because in the state that needs protection no validation filter exists. After the fix, a forged header alone is refused in both states, a valid token is still served,

and a valid token accompanied by a deliberately malformed forged header is still served — which demonstrates that the client’s value never reached the handler, since the handler fails closed on a malformed header. A regression test in the replication package asserts all of these.

Two properties are load-bearing for this design, and both must be stated wherever the offload is described. The mesh must overwrite the header, and mutual TLS must prevent any client from reaching the application port without traversing the filter chain. If mutual TLS is relaxed, the bypass reopens even with the filter applied.

6 Implications for practice

Pre-parse key material.

The single most effective change is to convert keys once at startup rather than per request. In our deployment this removed 98.1% of HS256 validation cost. It requires no architectural change, no new component and no change to the trust boundary.

Do not select algorithms from primitive benchmarks.

A primitive comparison ranks HS256 and RS256 by a wide margin; in deployment the gap is far smaller, because fixed per-call overhead dominates. Algorithm choice should weigh key distribution and trust-boundary properties, where the two differ substantially, rather than primitive speed.

Treat offloading as an architectural choice, not a remedy.

Once keys are pre-parsed, the remaining per-call cost is small. Offloading still

removes it, but it moves the trust boundary into the mesh and, as Section 5 shows, introduces a header-trust relationship that must be defended explicitly. That trade is worth making for reasons of policy centralisation; it is hard to justify on the basis of this overhead alone.

Measure in situ.

Our isolated benchmark understated deployed cost by more than an order of magnitude, and did so while being internally reproducible. Instrumenting the call inside the running service cost little and was the only measurement that exposed the error.

7 Threats to validity

Single runtime.

All measurements are Node.js. The key-conversion defect is a property of how this library is used, and other ecosystems may not share it. The methodological point — that isolated measurement mis-estimates deployed cost — is not runtime-specific, but our quantification of it is.

Single host, and contention.

Every measurement comes from one laptop-class machine on which the load generator, ingress, sidecars and service share cores. We record host load per run, and one sweep was discarded after background indexing was found to have perturbed it. As argued in Section 3, contention inflates in-situ duration rather than deflating it, so the central comparison is conservative. Absolute figures should not be compared against runs on dedicated hardware.

RS256 per-request cost is not verified in situ.

We measured RS256 inside the container and across a rate sweep, but the service was HS256-only for the configuration our earlier study evaluated, and we do not verify any per-request RS256 figure from that study. It remains unverified and we do not restate it.

Capacity is out of scope.

We make no claim about throughput ceilings, saturation points or autoscaling thresholds. Measurements bearing on capacity were attempted and discarded as unreliable on this host; only duration-distribution results, which are robust to contention in the direction described, are reported.

One application shape.

The endpoint returns a small JSON payload. Applications doing substantial per-request work would see validation as a smaller fraction, though the absolute per-call figures should carry over.

8 Conclusion and future work

The per-request cost of token validation in a cloud-native microservice is real and worth attending to, but it is not cryptographic. In our deployment the signature primitive is under one percent of it, and the dominant term is key material converted from a string on every call — a defect that a one-line change removes, and one that published evaluations, including our own, have measured without recognising. A secondary effect, the dependence of per-call cost on arrival rate, is large enough that a benchmark run in a tight loop understates realistic cost by more than an order of magnitude,

and we isolate the inter-arrival gap as its mechanism.

Future work should establish why per-call key conversion is so much more expensive under virtualisation than on the host, which we did not determine; extend the measurement to other runtimes and libraries to establish whether the defect generalises; and verify per-request RS256 cost in situ, which remains open.

Declarations

Data and code availability. All measurements, the instrumentation, the analysis pipeline and the scripts that produce every figure and numeric value in this paper are available in the replication package [Sikdar \(2026b\)](#). Every number in this paper is generated from a file under `data/runs/`; none is entered by hand.

Competing interests. The author declares no competing interests.

References

- Bremner Barr A, Lavi O, Naor Y, et al (2024) Technical report: Performance comparison of service mesh frameworks: the mTLS test case. arXiv preprint arXiv:2411.02267 ArXiv:2411.02267
- Gambo ML, Almulhem A (2025) Zero trust architecture: A systematic literature review. Journal of Network and Systems Management <https://doi.org/10.1007/s10922-025-09998-x>
- Hanada H, Ishibashi K (2025) Service-level objective-aware load-adaptive timeout: Balancing failure rate and latency in microservices. IEEE Access

- 13:137881–137895. <https://doi.org/10.1109/ACCESS.2025.3596118>
- Kabamba H, Khouzam M, Dagenais M (2026) Node compass: runtime-based multilevel tracing and debugging of single-threaded environments. *International Journal of Parallel, Emergent and Distributed Systems* <https://doi.org/10.1080/17445760.2026.2616774>
- Montenegro JA, Rios R, Bonilla J (2026) Comparative analysis of post-quantum handshake performance in QUIC and TLS protocols. *Computer Networks* 275:111957. <https://doi.org/10.1016/j.comnet.2025.111957>
- Mushtaq S, Mohsin M, Mushtaq M (2025) A systematic literature review on the implementation and challenges of zero trust architecture. *Sensors* 25:6118. <https://doi.org/10.3390/s25196118>
- OWASP Foundation (2023) OWASP API security top 10 – 2023. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, aPI2:2023 Broken Authentication
- Peng Y, Hao J, Chen Y (2025) Performance prediction and resource adaptive adjustment for cloud-native microservices. *Cluster Computing* 28:786. <https://doi.org/10.1007/s10586-025-05437-3>
- Schiavio F, Bulej L, Binder W (2026) Misleading microbenchmarks on the Java virtual machines. *arXiv preprint arXiv:260523570* ArXiv:2605.23570
- Sikdar K (2026a) Quantifying the performance cost of API security in cloud-native microservices: An empirical study of TLS, mTLS, and JWT validation. Unpublished manuscript; testbed and raw results at <https://github.com/Katha-Sikdar/microservices-api>. The per-request attribution in that study is corrected by the present paper.
- Sikdar K (2026b) Replication package: token-validation cost measurements. <https://github.com/Katha-Sikdar/microservices-api>, all measurements cited in this paper are under `psao-artifact/data/runs/`
- Singh S, Muntean CH, Gupta S (2025) Resilient microservices: an investigation into Istio effectiveness in Kubernetes. *Cluster Computing* 29:27. <https://doi.org/10.1007/s10586-025-05750-x>
- Souvatziadaki K, Limniotis K (2025) Post-quantum key exchange in TLS 1.3: Further analysis on performance of new cryptographic standards. *Cryptography* 9:73. <https://doi.org/10.3390/cryptography9040073>
- Traini L, Cortellessa V, Di Pompeo D, et al (2022) Towards effective assessment of steady state performance in Java software: are we there yet? *Empirical Software Engineering* 28. <https://doi.org/10.1007/s10664-022-10247-x>
- Tymoshenko I, Maraschi L, Collina M (2026) Predictive autoscaling for Node.js on Kubernetes: Lower latency, right-sized capacity. *arXiv preprint arXiv:260419705* ArXiv:2604.19705
- Zhu X, She G, Xue B, et al (2023) Dissecting overheads of service mesh sidecars. In: *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*, pp 142–157, <https://doi.org/10.1145/3620678.3624652>